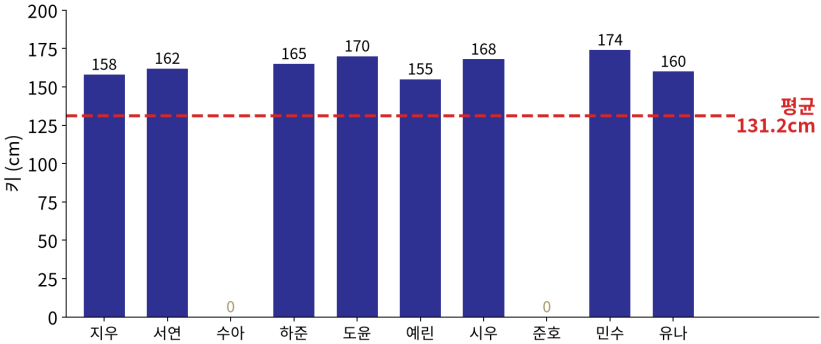
데이터 직접 다루기

더티 데이터 (결측치 · 중복 · 이상치) 와 정규화 · 표준화

Machine Learning and Deep Learning

KSA 2027-Spring
Sumin Han (suminhan@ksa.hs.kr)

오늘의 질문



우리 반 10명의 키. **평균 키가 131cm?** 뭔가 이상하다

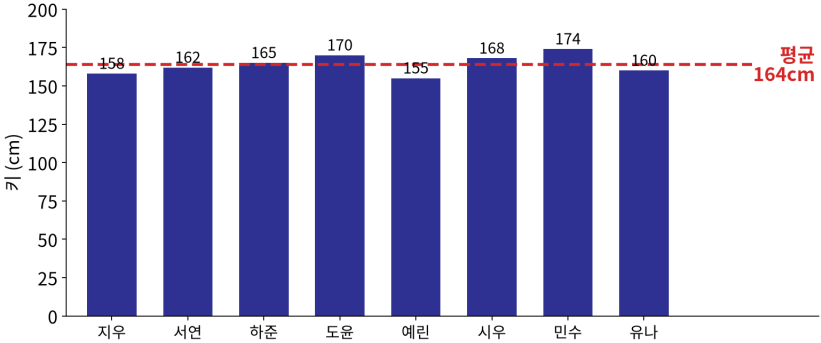
더티 데이터의 세 가지 모양

① 결측치	② 중복	③ 이상치
있어야 할 값이 없다	같은 기록이 여러 번	혼자 크게 튀는 값
Missing value	Duplicate	Outlier

모델을 고르기 전에, 데이터부터 본다

Garbage in, garbage out: 쓰레기를 넣으면 쓰레기가 나온다

① 결측치: 0을 빼면



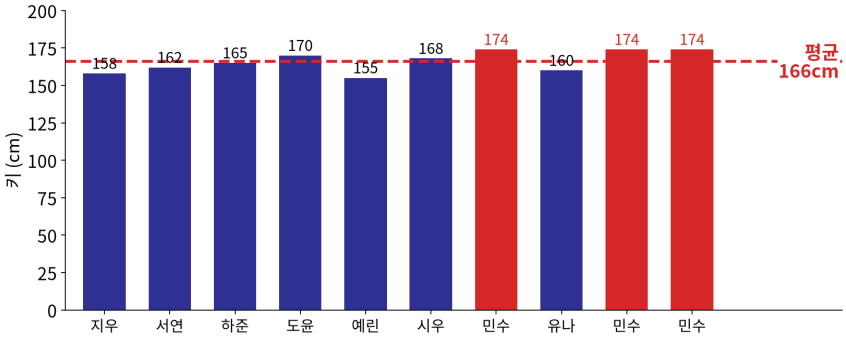
키를 안 적은 두 칸을 빼고 계산하면 **평균 164cm**

숫자인 척하는 결측치가 더 위험하다

적힌 값	컴퓨터가 보기에	
(빈칸)	비었음	**눈에 띈다**
NA, ?	글자	계산하다 오류가 난다
0, -999	**진짜 숫자**	**오류 없이 평균에 섞인다**

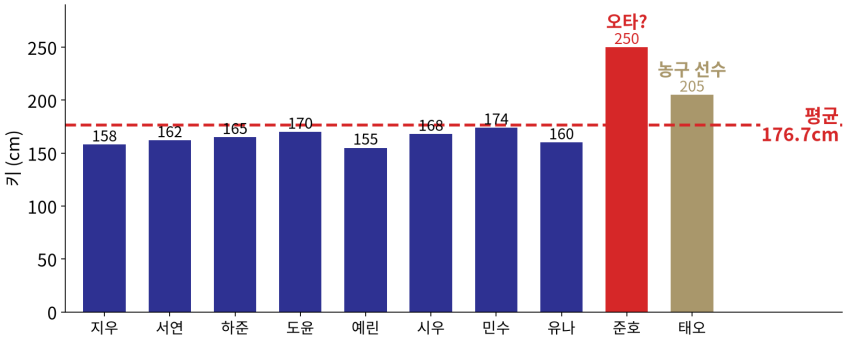
키 0cm, 나이 -999세처럼 **말이 안 되는 값**을 찾아서 빈칸으로 바꾼다

② 중복: 같은 기록이 여러 번



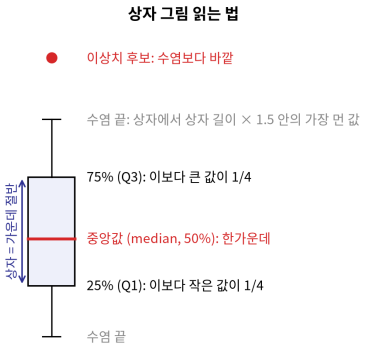
민수 기록이 세 번 들어가면 **평균 166cm** (원래 164cm)

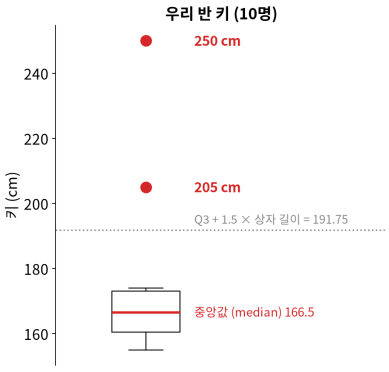
③ 이상치: 혼자 크게 튀는 값



준호 250cm는 **오타**, 태오 205cm는 **농구 선수**일 수도

그림으로 찾는다: 상자 그림





수염 밖에 홀로 떨어진 점이 이상치 후보

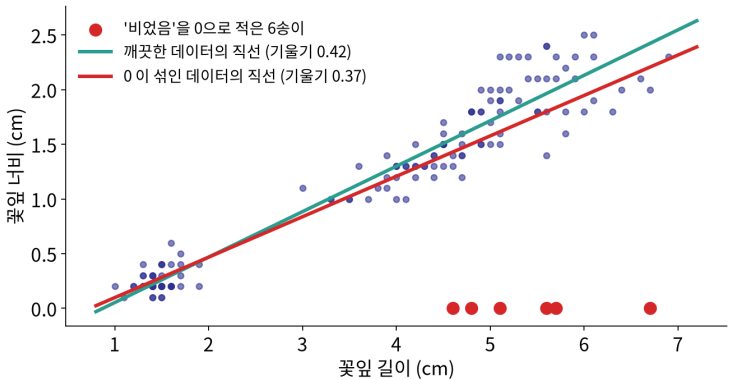
후보를 찾는 건 그림이, 지울지 남길지는 사람이 정한다

지울까, 고칠까, 남길까

경우	예	처리
분명한 오타	키 250cm	빈칸으로 (결측치처럼)
단위 실수	꽃받침 51cm (mm 로 적음)	고친다 ($\div 10$)
진짜일 수 있음	농구 선수 205cm	**남긴다**
모르겠음	?	표시해 두고, 지운 결과와 둘 다 보고

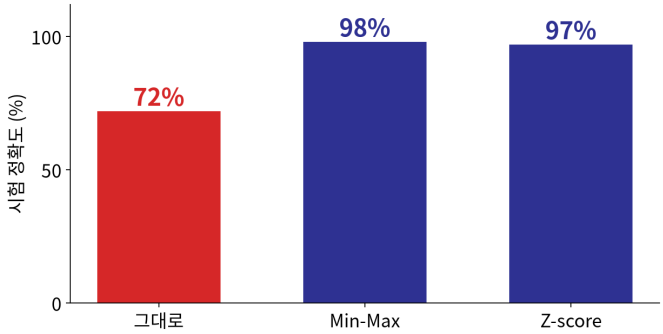
판단의 근거는 그 분야를 아는 지식 (도메인 지식)

모델도 속는다



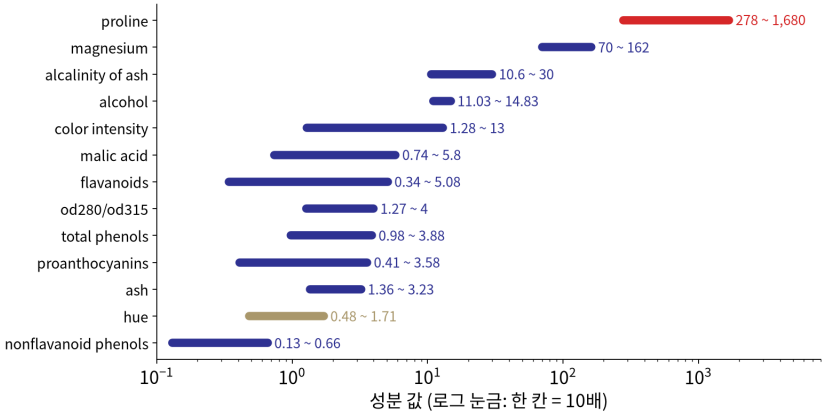
붓꽃 150송이 중 **6송이**의 꽃잎 너비를 0으로 적었을 뿐인데, 직선이 끌려 내려간다

두 번째: 값의 크기를 맞춘다



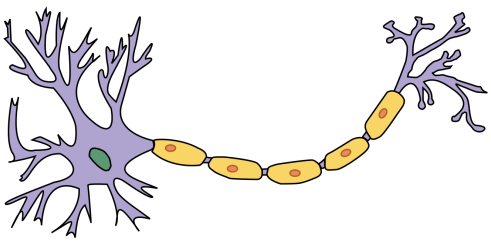
같은 신경망, 같은 와인 데이터. **성분 값을 그대로 넣으면 72%**, 정규화하면 **98%**

와인 한 병 = 성분 값 13가지, 범위가 제각각



프롤린 278 ~ 1,680, **색조 0.48 ~ 1.71**: 천 배 넘게 차이 난다

뉴런 하나가 하는 일



$$\text{output} = \tanh(w \cdot x + b)$$

x = 성분 값 13개 (데이터)

w = 뉴런이 **학습할 가중치** 13개

PyTorch `nn.Linear(13, 16)` 의 w 는 처음에
$-0.28 \sim 0.28$ 사이 작은 난수 ($\pm 1/\sqrt{13}$)

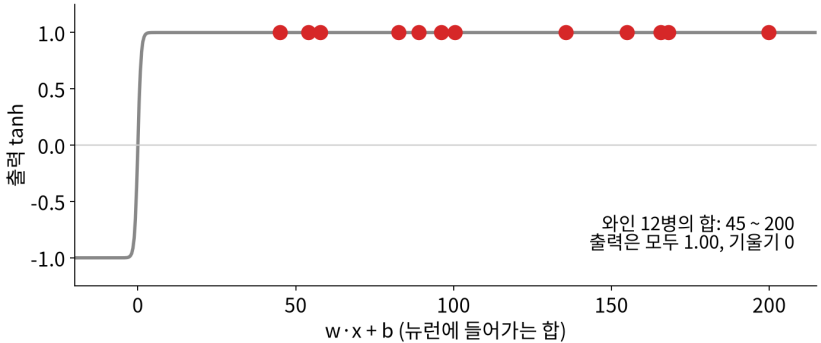
와인 1번 병을 그대로 넣으면

성분	값 x		가중치 w		$x \times w$
프롤린	**1,065**	$\times$	0.15	$=$	**162.15**
마그네슘	127	$\times$	0.05	$=$	6.56
알코올	14.23	$\times$	0.11	$=$	1.57
색조	1.04	$\times$	-0.16	$=$	-0.17
나머지 9개 $+ b$					-1.98
합 $w \cdot x + b$					**168.13**

$\tanh(168.13) = \mathbf{1.00}$ 프롤린 하나가 합을 다 차지한다

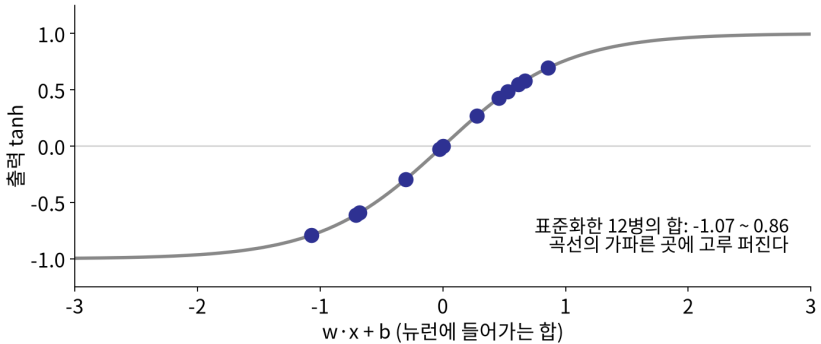
w 는 반올림해 적었고, 곱은 반올림 전 값으로 계산 (seed 0, 4번 뉴런)

12병 모두 출력 1.00: 기울기 0, 배우지 못한다



곡선이 평평한 곳 = **기울기 0**. 학습 전 뉴런 출력의 **98%**가 이렇게 막혀 있다

성분마다 분포를 맞추면



프롤린 1,065 → **1.01**. 합 **0.67**, 출력 **0.58**: 곡선의 가파른 곳에서 신호가 산다

방법 1. Min-Max 정규화: 0 ∼ 1 로

$$x' = \frac{x - \min}{\max - \min}$$

성분마다 따로: 가장 작은 값 → 0, 가장 큰 값 → 1

프롤린 1,065: $\frac{1065 - 278}{1680 - 278} = \frac{787}{1402} = \mathbf{0.56}$

쓰기 좋은 곳: 이미지 픽셀처럼 **범위가 정해진 값** (0 ∼ 255 → 0 ∼ 1)

방법 2. Z-score 표준화: 평균 0, 표준편차 1 로

$$z = \frac{x - \mu}{\sigma}$$

μ = 평균 (mean), σ = 표준편차 (standard deviation), 성분마다 따로

프롤린 1,065: $\frac{1065 - 747}{314} = \mathbf{1.01}$

"평균보다 **표준편차 1개만큼** 많다"

보통은 이것을 쓴다. 튀는 값 하나가 있어도 Min-Max 보다 덜 흔들린다

평균 · 표준편차는 학습 데이터로만 구한다

단계	할 일	
학습 데이터	μ, σ (또는 min, max) 를 **구한다**	`fit`
학습 데이터	그 값으로 바꾼다	`transform`
시험 · 새 데이터	**같은** μ, σ 로 바꾼다 (새로 구하지 않는다)	`transform`

시험 데이터로 평균을 구하면 **시험 문제를 미리 엿본 것** (데이터 누수)

회귀에서 정답 y 도 표준화했다면, 예측값은 마지막에 $\hat{y} = \hat{z}\,\sigma_y + \mu_y$ 로 원래 단위로 되돌린다

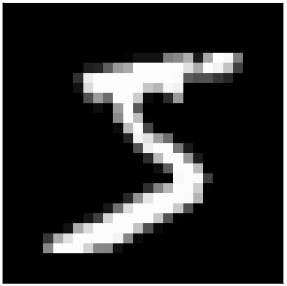
데이터마다: “무엇마다” 맞추나

데이터	모양	무엇마다	보통 하는 일
표 (tabular)	(행, 열)	**열(성분)마다**	Z-score
흑백 이미지	(H, W)	이미지 전체	$\div 255$, 그다음 평균 · 표준편차
컬러 이미지	(H, W, 3)	**채널(R, G, B)마다**	$\div 255$, 채널별 평균 · 표준편차
언어	단어의 줄	글자 표기	표기 맞추기, 번호는 **그대로**
시계열	(시간,)	**과거만**, 또는 창마다	학습 기간 Z-score, 창마다 Z-score
소리	(샘플,)	녹음마다	$\div 32768$, 음량 맞추기, dB
비디오	(T, H, W, 3)	채널마다, **영상 전체로**	이미지와 같게, 프레임마다 따로는 X

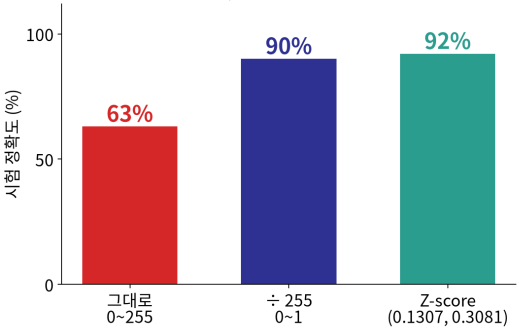
먼저 **데이터를 열어서** 모양 · 자료형 · 최솟값 · 최댓값을 본다

흑백 이미지: 픽셀 0 ∼ 255 를 그대로 넣으면

28 × 28 픽셀, 값 0 ~ 255 (uint8)



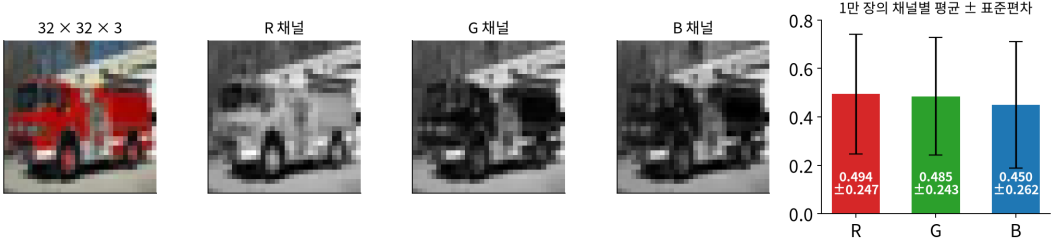
MNIST, 같은 신경망 · 같은 학습



MNIST: $\div 255$ 만 해도 **63%** → **90%**. 그대로면 학습 전 뉴런 **95%**가 막힌다

학습 1만 장 · 시험 1만 장, MLP 784-64-10 (tanh), SGD 0.1, 3에폭, 시드 3개 평균. 0.1307, 0.3081 = 학습 6만 장의 픽셀 평균 · 표준편차

컬러 이미지: 채널(R, G, B)마다 따로



CIFAR-10: 평균 R **0.494**, G **0.485**, B **0.450**. 채널마다 빼고 나눈다

사진은 (H, W, C), PyTorch 는 **(C, H, W)**: 축 순서부터 확인

남이 학습시킨 모델은 그 모델의 값을 그대로 (ImageNet: 0.485, 0.456, 0.406)

언어: 숫자 크기가 아니라 표기를 맞춘다

① 표기 맞추기

FREE, Free, free! → free
SMS 5,574통의 단어 종류 **15,733** → **8,713**

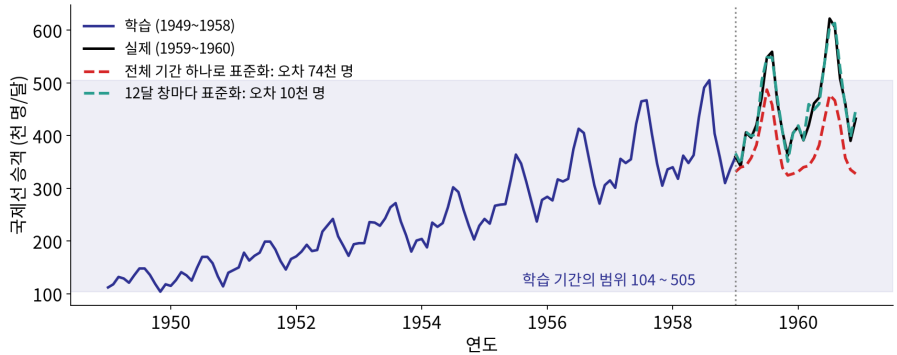
② 단어 번호는 크기가 아니다

free $= 3373$, call $= 1828$: 3373 이 "더 큰" 단어는 아니다
→ 번호를 표준화하지 않는다. 임베딩 벡터로 바꾼다
(nn.Embedding 은 처음에 평균 0, 표준편차 1 난수)

③ 단어 세기는 길이로 나누기

1단어 문자와 171단어 문자: 긴 글일수록 숫자가 커진다

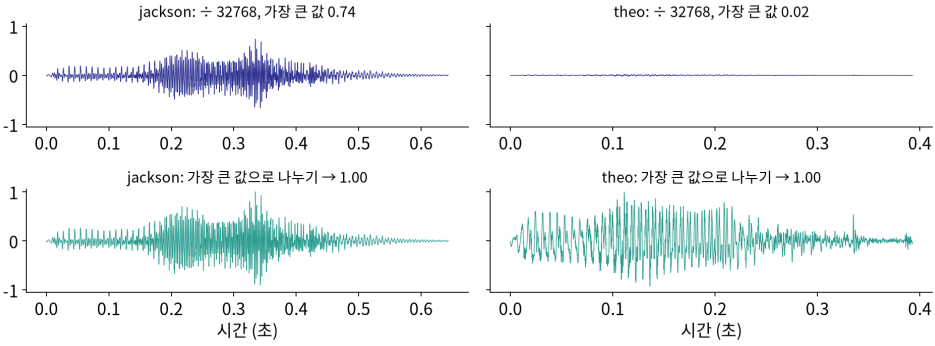
시계열: 미래는 과거의 범위를 넘는다



학습 최대 505 → 시험 **622** (Min-Max **1.29**) 창마다 표준화: 오차 **74** → **10**천 명

국제선 항공 승객 1949∼1960, 앞 10년 학습 · 뒤 2년 시험, 지난 12달 → 다음 달, k-최근접 이웃 ($k = 3$)

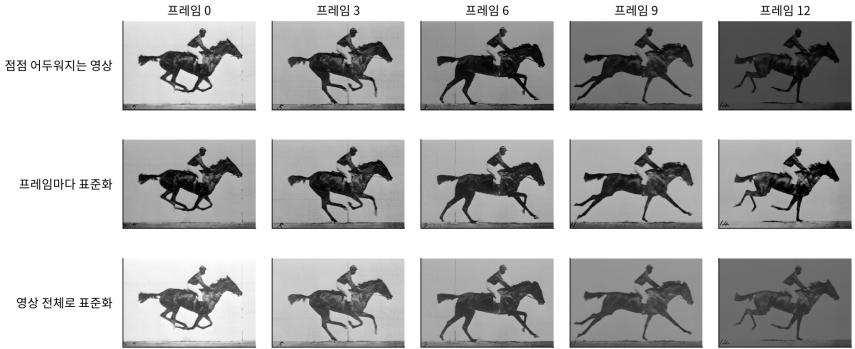
소리: 녹음마다 음량이 다르다



같은 “zero”, 음량 **37배** 차이. int16 $\div 32768 \to -1 \sim 1$, 그다음 **녹음마다** 맞춘다

Free Spoken Digit Dataset, 8,000 Hz. 가장 큰 값으로 나누기 = 피크 정규화 (녹음마다 표준편차로 나누는 방법도 쓴다)

비디오: 이미지 × 시간, 영상 전체로 맞춘다



(T, H, W, C) = (15, 200, 300, 3). 프레임마다 표준화하면 **어두워지는 변화가 사라진다**

Muybridge, The Horse in Motion (1878). 실습에서는 일부러 점점 어둡게 만든 영상으로 비교

문제풀이

1. 키 기록 `[160, 0, 170, 0, 180]`. 평균은? 0을 결측치로 보고 빼면?
2. 다음 값은 오타일까, 진짜일까? 무엇을 더 알아야 할까? (a) 고등학생 몸무게 450kg (b) 하루 공부 시간 25시간
3. 8명의 하루 공부 시간 `[2, 4, 4, 4, 5, 5, 7, 9]`. 평균과 표준편차(n 으로 나눔)는? 9시간의 Z-score 는? 5시간의 Min-Max 값은?
4. 학습 데이터의 키 최솟값 150, 최댓값 190 으로 Min-Max 를 만들었다. 시험 데이터에 키 200 이 나오면 값은? 시험 데이터로 min, max 를 다시 구해야 할까?
5. `np.uint8(10) - np.uint8(20)` 은 얼마일까? 컬러 이미지 `X.shape == (10000, 32, 32, 3)` 의 채널별 평균은 `X.mean(axis=?)` ?

실습: 데이터를 직접 열어 보기

1. 더티 데이터: 일부러 더럽힌 iris 청소 (빈칸 · 0 · -999, 표기, 51cm 꽃받침) **86%** → **96%**
2. 표: 와인 13성분, Min-Max · Z-score 구현 → **72%** vs **98%**
3. 흑백 이미지: MNIST uint8 함정, $\div 255$, 평균 · 표준편차
4. 컬러 이미지: CIFAR-10 채널별 평균 · 표준편차, (H, W, C) → (C, H, W)
5. 언어: SMS 표기 맞추기 (15,733 → 8,713), 단어 세기 길이로 나누기
6. 시계열: 항공 승객, 학습 기간으로만 vs 창마다 표준화 (**74** → **10**)
7. 소리: 숫자 말소리 6명, $\div 32768$, 음량 맞추기, dB 스펙트로그램
8. 비디오: 달리는 말 GIF, 프레임마다 vs 영상 전체

데이터는 첫 셀에서 내려받기 (MNIST · CIFAR-10 은 수십 MB)

▶ Colab에서 실습 노트북 열기

정리

결측치는 0, -999 처럼 **숫자인 척**한다. 이상치는 그림으로 찾고 판단은 사람이

값의 크기가 제각각이면 뉴런이 **막힌다** (tanh 출력 1.00, 기울기 0)

Min-Max $\frac{x - \min}{\max - \min}$ Z-score $\frac{x - \mu}{\sigma}$ 통계는 **학습 데이터로만**

무엇마다 맞출지는 데이터가 정한다: 열 · 채널 · 녹음 · 창 · 영상 전체

어떤 데이터든 먼저 **열어서** 모양 · 자료형 · 최솟값 · 최댓값을 본다

이미지 · 데이터 출처

- 뉴런 그림: Wikimedia Commons “Neuron.svg” (Dhp1080, CC BY-SA 3.0), 라벨 선을 지워 사용
- 달리는 말: Eadweard Muybridge, The Horse in Motion (1878), 퍼블릭 도메인 (Wikimedia Commons)
- 붓꽃: Iris (Fisher, 1936; scikit-learn 내장) 와인: UCI Wine (Aeberhard & Forina, 1991)
  손글씨 숫자: MNIST (LeCun, Cortes, Burges)
- 컬러 사진: CIFAR-10, A. Krizhevsky (2009) “Learning Multiple Layers of Features from Tiny Images”
- 문자 메시지: UCI SMS Spam Collection (Almeida & Hidalgo, 2011)
- 항공 승객: Box & Jenkins, Time Series Analysis (1976), 1949∼1960 국제선 월별 승객
- 말소리: Free Spoken Digit Dataset (Z. Jackson 외, CC BY-SA 4.0)